Chapter 4

Part 1: Overview

The lecture and notebook build deep learning mechanics from scratch- moving from raw data representations to calculus optimization and multi-layer neural networks:

- **Model Architectures & timm Library:**
  - Moving beyond basic ResNets to modern computer vision architectures like ConvNeXT
  - Evaluating model trade-offs across speed (seconds per sample), memory usage, and accuracy on ImageNet.
  - Understanding the benefit of pre-training on 22,000 ImageNet categories (in22ft1k)
- **Deconstructing Serialized Models (model.pkl):**
  - Unpacking what lives inside a fastai Learner object: data preprocessing pipelines and trained model parameters weights.
  - Navigating PyTorch module hierarchies using get-submodule to inspect individual layer weights and normalization layers directly.
- **Image Representation, Tensors & Broadcasting:**
  - How computers store greyscale images 28 x 28 numerical pixel grids (values from 0 to 255)
  - Understanding tensor rank (number of axes) versus tensor shape (length of each axis)
  - Leveraging broadcasting to perform elementwise array operations across thousands of images simultaneously at C/CUDA speed without explicit Python loops.

Chpt 4 continued

Part 1

- **The Baseline Model (Pixel Similarity):**
  - Constructing composite "ideal" digits by averaging pixel grids across train samples.
  - Measuring grid-to-grid distances using L1 norm (mean absolute difference) and L2 Norm / MSE / RMSE (Mean Squared Error / Root Mean Squared Error).
- Optimization via Stochastic Gradient Descent (SGD):
  - The 7-step optimization loop: Initialize → Predict → Loss → Gradient → Step → Repeat → Stop
  - Why discrete metrics (like accuracy) cannot drive training because their gradients are zero almost everywhere, requiring smooth, differentiable loss functions (like mnist_loss with sigmoid).
  - Automated differentiation using PyTorch's `<requires_grad_()>` and `<loss.backward()>`
  - Stepping parameters using a learning rate (lr) hyperparameter, and bundling data into mini-batches via DataLoaders for GPU parallelization and stable gradient steps.
- **Neural Network Mechanics & Non-Linearities (ReLUs)**
  - Why stacking multiple linear layers ($y = X \cdot W + b$) without activation functions collapses mathematically into a single linear layer.
  - Introducing the Rectified Linear Unit (ReLU) ($\max(0, x)$) as an activation function that decouples linear layers.
  - Universal Approximation Theorem: proving that combining linear transformations with non-linear ReLUs allows a neural network to fit any arbitrary complex mathematical function.
  - Building multi-layer models (w/ `<nn.Sequential>`) and train with fastai learner.

Chapter 4 continued

Part 1

- **Matrix Multiplication & The Excel Spreadsheet Model:**
  - Demonstrating how matrix multiplication ($x \cdot W + b$) serves as the computational engine running across GPU Tensor Cores.
  - Building a complete neural network from scratch inside a spreadsheet on the titanic dataset - handling dummy variables ($N-1$ rule), continuous feature normalization, constant bias columns, and optimizing weights using Excel Solver.
- **Introduction to Natural Language Processing (NLP):**
  - Transitioning from vision and tabular data to text classification.
  - Previewing Hugging Face Transformers, document tokenization, and validation set design.

Chapter 4

Part 2: High-Level Conceptual Deep Dive

1. The Grid Representation

A computer has no inherent human understanding of visual shapes like a "3", an "8", or a "4". It only understands numbers. In computer vision, a grayscale image is converted into a 28 × 28 numerical grid where every single cell contains an intensity value from 0 (pure white background) to 255 (fully inked black).

e.g. of snippet "raw 28 × 28 pixel grid values"

0	0	0	0	0	0	
0	0	29	150	195	254	
0	48	224	253	253	234	
0	93	244	249	253	187	
0	107	253	253	230	48	

2. Building "Ideal Digit" Baselines

To build a simple baseline model, we collect thousands of handwritten images for a specific digit (e.g. 6131 sample images of the number 3) and stack them into a 3D tensor shape.

We then take the average along the image axis (mean(0)). This computes the average pixel intensity for every single coordinate position across all 6,131 images. The result is a single 28 × 28 grid representing the blurry, composite "ideal 3". We repeat this process to generate an "ideal 7" grid, and so on. Where all training images agreed ink should be present and: ideal grid positions are dark; and where they disagreed they are wispy and light.

Chapter 4

Part 2 continued

3. Classifying a New Image via Distance (MSE/RMSE/L1)

When a new handwritten image is fed into the system, it is converted into its own sharp 28 × 28 pixel grid. To classify it, the model compares this new input grid position-by-position against every pre-computed ideal grid.

Because direct subtraction would allow positive pixel differences to cancel out negative differences, we must collapse all 784 pixel comparison differences into a single distance score using a standardized metric:

- L1 Norm (Mean Absolute Difference): `(a-b).abs().mean()` - compute the average absolute pixel difference across all cordinates.
- L2 Norm / MSE / RMSE (Mean Squared Error / Root Mean Squared Error): `((a-b)**2).mean().sqrt()` - takes the difference at each pixel position, squares it, calculates the mean acrosss all pixels, and takes square root.

Because MSE squares every pixel difference before averaging, it penalizes large pixel disagreements much more heavily than small ones. Whichever ideal grid yields the smallest MSE distance to the input image wins the prediction. The baseline model is simply asking: "Which pre-computed ideal average grid does this new pixel grid sit closest to in space?"

Chapter 4

Part 2 continued

4. The OCR Misclassification Analogy (The Sloppy "4" vs "9")

The grid comparison explains why Optical Character Recognition (OCR) systems make mistakes on bad handwriting. If you write a 4 with sloppy handwriting and an OCR model classifies it as a 9:

- The model did not "misunderstand" your handwriting conceptually.
- Pixel-for-pixel, the spatial ink layout of your sloppy 4 landed numerically closer to the trained "ideal 9" grid than to the "ideal 4" grid.
- If your handwritten 4 had a slightly curved or close loop at the top, those dark pixels fell directly into coordinate positions where the training set's 9s usually contained dark ink. Simultaneously, if your strokes missed the sharp vertical alignment expected in a standard 4, those missing pixels added distance penalty against the ideal 4 grid.
- By the raw numbers, your grid sat closer to the 9 average than the 4 average.

Chapter 4

Part 3: Deep Dive - Why Baselines Fall Short

The Fundamental bottleneck of Pixel Similarity

While the pixel-similarity baseline achieves ~95% accuracy on simple two-digit problems (like distinguishing a 3 from a 7), it hits a hard ceiling because it cannot actually learn or adapt.

1. **Static Memorization**: It calculates fixed pixel averages once and remains completly frozen.
2. **No Feature Importance**: It treats every pixel coordinate equally. It cannot learn that pixel (14, 14) at the center of the image might be 100 times more critical for distinguishing digits than a background pixel in the top-left corner.
3. **Spatial Rigidity**: If a perfectly drawn "3" is shifted just 2 pixels to the left, its active pixels no longer align with the dark center of the ideal "3" grid, causing the pixel-by-pixel distance to explode.

1. Replacing Static Averages with Trainable Weights (W) and Biases (b)

To convert this baseline into a true learning model, we replace the static "ideal average grids" with trainable weight parameters (W), biases (b)

Instead of comparing raw pixel values directly against fixed averages, every input pixel $(x_1, x_2, \dots x_{784})$ is multiplied by its own corresponding learnable weight parameter $(w_1, w_2 \dots w_{784})$.

Chapter 4
Part3 continued

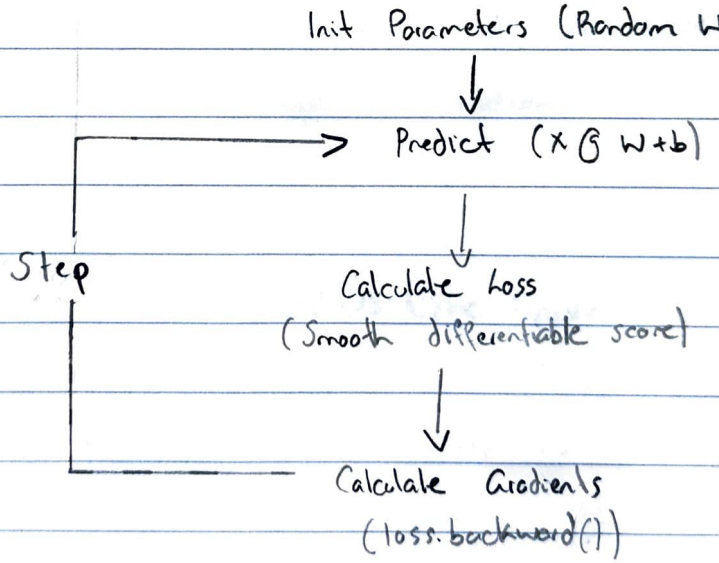
$$\text{Prediction Score} = (x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_{784})$$

- Weights (W): Flexible, trainable feature detectors. A highly positive weight means "dark ink at this pixel strogly increases the probability of this category." A negative weight means "dark ink at this pixel actively penalizes this category!"
- Bias (b): Acts as a trainable threshold or intercept, giving the model an overall baseline tendency to predict a class regardless of pixel values.

2. The egine of learning: Loss Functions & SGD

Rather than guessing manually weight values, we use Stochastic Gradient Descent (SGD) to let the computer learn the optimal weights automatically.

e.g.



1. Continuous Signal vs. Discrete Metrics: We cannot use discrite classification accuracy to train weights because a tiny tweak to a weight value almost never flips a discrete prediction (e.g. changing a prediction from 0.49 to 0.499 still evaluates as "False"), resulting in zero gradients everywhere.

Chapter 4

Part 3 continued

2. Smooth Loss Function: We pass raw model scores through a Sigmoid function ($1/(1+e^{-x})$) to squash scores into smooth probabilities between 0 and 1. We then calculate a smooth loss function (mnist_loss) that rewards confident correct predictions and penalizes incorrect or hesitant predictions.

3. Gradients & Automated Backpropagation: Calling loss.backwards() calculates the calculus derivative (gradient) for every single weight. The gradient tells us the exact slope of the error surface: "If I increase weight w_{14} by a tiny amount, will the total loss go up or down, and by how much.

4. Optimizer Step: We update every weight parameter by taking a small step in the opposite direction of its gradient, scaled by the learning rate (lr) hyperparameter ($W = W - lr \cdot grad$).

3. Breaking Linearity with ReLUs (The Universal Approximation Theorem)

A single weight matrix ($X \cdot W + b$) is purely linear. It can only draw straight decision planes across the 784-dimensional pixel space. If we stack two linear layers directly on top of each other, the math collapses back into a single linear transformation:

$$y = (X \cdot W_1 + b_1) \cdot W_2 + b_2 = X \cdot (W_1 \cdot W_2) + (\dots$$

Stacking 100 linear layers without non linearities has no more learning power than a sigle linear layer.

Chapter 4

Part 3 continued

To unlock the true power of deep learning, we insert a Rectified Linear Unit (ReLU) - defined as $\max(0, x)$ - between linear layers.

```
0  simple_net = nn.Sequential( nn.Linear(28*28, 30)
1  # above Layer 1 extract 30 linear combinations
2  nn.ReLU()
3  # above Non linearity truncate negative values to 0
4  nn.Linear(30, 1)
5  # above Layer 2: combine activation features into final score
```

By clipping negative activation values to zero, the ReLU acts as a conditional threshold ("if value > 0"). According to the Universal Approximation Theorem, combining linear matrix multiplication with non-linear ReLUs allows a neural network to construct arbitrarily complex, "wiggly" multi-dimensional decision boundaries. With enough hidden units and parameters, it can approximate any continuous mathematical function.

Chapter 4

Part 3 continued

4. From Rigid Pixel Matching to Hierachical Feature Extraction

This transition from baseline averages to deep neural networks completly changes how the computer views an image:

- The Baseline Model: Asks "Does pixel (14, 14) in my input match pixel (14, 14) in the ideal '4' grid?
- Deep Neural Networks:
  - Layer 1 weights multiply raw pixels to detect baric primative features (slanted lines, horizontal edges, sharp corners).
  - The ReLU activation turns features on and off based on wheter those primatives are present.
  - Layer 2 & deeper layers combine the basic primatives into higher-level abstract concepts (e.g. "a vertical stroke intersecting a horizontal crossbar at a 90-degree angle")

This solves our OCR problem: a deep neural network is no longer fooled by a slighty shifted or sloppy handwritten 4 because it is no longer performing a rigid coordinate-by-coordinate pixel overlay. Instead, it evaluates the hierachical presence and structral relationship of visual features, allowing it to correctly classify imges across variations in handwriting, scale, rotation, and alignment.